

DATA ANALYTICS

IT ACADEMY | BCN ACTIVA

SPRINT 07

Estructuras de datos y de control
16 de junio de 2025



Letiane Benincá
benincalf@gmail.com

IT ACADEMY



TAREA S7.01

ESTRUCTURAS DE DATOS Y DE CONTROL.

Objetivos:

- Comprender y familiarizarse con las estructuras básicas de datos y control en Python.

Duración: 3 días.

Entrega: Almacena en un repositorio de tu GitHub una carpeta que contenga:

- El archivo Jupyter Notebook (.ipynb) que contenga cada enunciado acompañado por los códigos de las respuestas y su respectiva ejecución.
- En la entrega, coloca el enlace al repositorio.

DESCRIPCIÓN

Resolverás algunos problemas de la vida cotidiana aplicando las estructuras de datos y control en Python.

Un cliente de la empresa en la que trabajas solicita una lista de programas muy sencillos, pero que le facilitarían muchos procesos. Sin embargo, el departamento de TU (Tecnología/Informática) está muy saturado de trabajo, por lo que te piden que realices la programación.

NIVEL 1

1. **Calculadora del índice de masa corporal (IMC):** Escribe una función que calcule el IMC introducido por el usuario, es decir, quien lo ejecute deberá ingresar dichos datos. Puedes obtener más información sobre su cálculo en: [Índice de masa corporal \(IMC\): qué es y cómo se calcula](#). La función debe clasificar el resultado en sus respectivas categorías.

$$\text{Cálculo: } IMC = \frac{\text{Peso (Kg)}}{\text{Altura (m)}^2}$$

- a) Bajo peso: menos de 18,5.
- b) Peso normal: 18,5-24,9.
- c) Sobrepeso: 25-29,9.
- d) Obesidad: más de 30.

```
def calcular_imc():
    # Input de los datos:
    peso = float(input('Introduzca su peso en Kg (para decimales, utilizar punto): '))
    altura = float(input('Introduzca su altura en metros, (para decimales, utilizar punto): '))

    # Cálculo:
    imc = peso / (altura ** 2)

    # Realizar la verificación de los rangos de imc:
    if imc < 18.5:
        print('Bajo de peso')
    elif 18.5 <= imc < 24.9:
        print('Peso Normal')
    elif 25 <= imc < 29.9:
        print('Sobrepeso')
    else:
        print('Obesidad')

    # Devolver o resultado do cálculo:
    return (imc)

resultado = calcular_imc()
print(f'El resultado del cálculo de su IMC es de: {resultado:.2f}')
```

✓ 4.8s

Peso Normal

El resultado del cálculo de su IMC es de: 20.43

Explicación de la función: Creamos la función `calcular_imc`. Primero, solicitamos al usuario que ingrese su peso y altura mediante entradas de texto, usando `float` para poder utilizar números decimales. Luego, dentro de la función se realiza el cálculo del IMC dividiendo el peso entre la altura al cuadrado. Según el resultado del IMC obtenido, se 'printa' el mensaje indicando el rango en que se encuentra la persona. Por último, se imprime el valor del cálculo mediante la función `print`.

Después de la corrección con Fernando, hemos ajustado el código para que sea más legible en los condicionales y separamos la función de su llamada de resultado:

```
def calcular_imc():
    # Input de los datos:
    peso = float(input('Introduzca su peso en Kg (para decimales, utilizar punto): '))
    altura = float(input('Introduzca su altura en metros, (para decimales, utilizar punto): '))

    # Cálculo:
    imc = peso / (altura ** 2)

    # Realizar la verificación de los rangos de imc:
    if imc < 18.5:
        print('Bajo de peso')
    elif imc < 24.9:
        print('Peso Normal')
    elif imc < 29.9:
        print('Sobrepeso')
    else:
        print('Obesidad')

    # Devolver o resultado do cálculo:
    return (imc)

[1] ✓ 0.0s

resultado = calcular_imc()
print(f'El resultado del cálculo de su IMC es de: {resultado:.2f}')

[2] ✓ 7.9s

...
Peso Normal
El resultado del cálculo de su IMC es de: 20.78
```

También se ha comentado de utilizar algún tipo de verificación, caso el usuario inserta un valor diferente en `input`, como una `string` o un número fuera de rango. Pero, en este momento, preferí dejarlo sin esta verificación.

2. Convertidor de temperaturas: Existen varias unidades de temperatura utilizadas en diferentes contextos y regiones. Las más comunes son Celsius (°C), Fahrenheit (°F) y

Kelvin (K). También existen otras como Rankine (°Ra) y Réaumur (°Re). Selecciona al menos 2 convertidores, de tal forma que al introducir una temperatura devuelva, como mínimo, dos conversiones.

```
def conversor_temperaturas():
    try:
        valor = float(input('Introduzca el valor de la temperatura: '))
        escala = input('Introduzca la escala (C, F o K): ').upper()

        # Convertir todo a Celsius primero
        if escala == 'C':
            celsius = valor
        elif escala == 'F':
            celsius = (valor - 32) * 5/9
        elif escala == 'K':
            celsius = valor - 273.15
        else:
            print('Inválida')
            return

        # Convertir desde Celsius a las otras escalas
        fahrenheit = celsius * 9/5 + 32
        kelvin = celsius + 273.15

        print(f'La temperatura en Celsius es de: {celsius:.2f}° C')
        print(f'La temperatura en Fahrenheit es de: {fahrenheit:.2f}° F')
        print(f'La temperatura en Kelvin es de: {kelvin:.2f} K')

    except ValueError:
        print("Valor inválido")
```

✓ 0.0s

```
conversor_temperaturas()
```

✓ 4.8s

La temperatura en Celsius es de: 10.50° C
 La temperatura en Fahrenheit es de: 50.90° F
 La temperatura en Kelvin es de: 283.65 K

Explicación de la función: Primero, se ha definido la función llamada: `conversion_temperaturas`. Dentro de la función pedimos los inputs del usuario, para el valor de la temperatura (float) y la escala de esta temperatura (C, F o K), para no equivocarse, utilizamos el `,upper()` para que transforme los valores de entrada a mayúsculas. Después, convertimos el valor a Celsius, utilizando condicionales y si no entra en las escalas definidas, la salida será 'Inválida', terminando la función. Luego, convertimos de Celsius, para Fahrenheit y para Kelvin, así imprime los

resultados de las tres escalas de temperatura y luego, finaliza el try-except, manejando errores, por si el usuario añade algún numero que no es válido, enseña un mensaje de error.

- 3. Contador de palabras de un texto:** Escribe una función que, dado un texto, muestre cuántas veces aparece cada palabra.

```
def contar_palabras():
    # Pedir un texto:
    texto = input('Escriba un texto corto aquí: ')
    print(texto)

    # Dividir las palabras del texto:
    palabras = texto.lower().split()

    # Crear un diccionario vacío para almacenar el conteo:
    conteo_palabras = {}

    # Iterar sobre cada palabra en la lista:
    for palabra in palabras:
        palabra = palabra.strip('!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~') # Podría ser con la librería string (string.punctuation)
        if palabra in conteo_palabras:
            conteo_palabras[palabra] += 1
        else:
            conteo_palabras[palabra] = 1

    # Devolver el diccionario con el conteo de palabras
    return conteo_palabras

[4] ✓ 0.0s

resultado = contar_palabras()
print('Conteo de palabras:', resultado)

[5] ✓ 3.4s

... No importa lo lento que vayas, mientras no te detengas. Confucio.
Conteo de palabras: {'no': 2, 'importa': 1, 'lo': 1, 'lento': 1, 'que': 1, 'vayas': 1, 'mientras': 1, 'te': 1, 'detengas': 1, 'confucio': 1}
```

Explicación de la función: Se crea a función contar_palabras, que no recibe ningún parámetro. Luego, pedimos al usuario para escribir un texto, usando el input(). En la función, pedimos que se imprima el texto. Para contar las palabras, primero se convierte todo el texto a minúsculas con método. lower() e luego se divide en palabras con el método. split(). Utilizando un bucle for para recorrer cada palabra del texto. Inicialmente, se usa el método. strip dentro del bucle para eliminar posibles puntuaciones, luego va contando las palabras, si está ya está en el diccionario conteo_palabras, le sumamos 1, si no, la añadimos al diccionario y vinculamos su valor. Para finalizar la función, utilizamos el return y para llamar la función creamos una nueva variable (resultado), que será impreso en el print final, presentando el resultado final en su diccionario.

- 4. Diccionario inverso:** Resulta que el cliente tiene una encuesta muy antigua que se almacena en un diccionario y necesita los resultados al revés, es decir, intercambiando las claves y los valores. Las claves y valores en el diccionario original son únicos; si no lo son, la función debe imprimir un mensaje de advertencia.

```
reverse_dictionary({'a': 1, 'b': 2, 'c': 3})
```

```
{1: 'a', 2: 'b', 3: 'c'}
```

```
reverse_dictionary({'x': 'apple', 'y': 'banana', 'z': 'banana'})
```

```
Error: multiple keys for one value
```

```
def reverse_dictionary(diccionario):
    # Verificar si hay valores duplicados
    if len(diccionario.values()) != len(set(diccionario.values())):
        print('Error: multiple keys for one value')
    else:
        # Invertir el diccionario
        reversed_dict = {value: key for key, value in diccionario.items()}
        return reversed_dict
```

```
reverse_dictionary({'a': 1, 'b': 2, 'c': 3})
```

```
{1: 'a', 2: 'b', 3: 'c'}
```

```
reverse_dictionary({'x': 'apple', 'y': 'banana', 'z': 'banana'})
```

```
Error: multiple keys for one value
```

Explicación de la función: Se crea la función `reverse_dictionary`, que recibe un parámetro (diccionario). El objetivo es invertir el diccionario que se le asignará más tarde. Inicialmente, para asegurarnos de que no haya valores repetidos, porque si existen, al invertir, se perderá la información repetida. Usado el método `.values()`, obtenemos todos los valores del diccionario. Luego, comparamos con el `len`, cuantos valores hay y con el `set` (que elimina duplicados). Si las longitudes son diferentes, si identifica un valor repetido. En este caso si imprime el mensaje: 'Error: multiple keys for one value'. En el condicional `else`, damos el comando para invertir el diccionario, caso este no tenga duplicados. Esto crea un nuevo diccionario invirtiendo la clave por el valor (`value: key for key, value in diccionario.items()`). Por último, finalizamos la función dando el `return`

reversed_dict, que es el diccionario final, con los valores y las llaves invertidas. Para verificar los valores en pantalla, primero usamos el print, con la función y le presentamos el diccionario que tiene que invertir los datos. Y, por último, invocamos la función con el diccionario y así verificamos los dos diccionarios presentados en este ejercicio.

NIVEL 2

- 1. Diccionario inverso con duplicados:** Continuando con el ejercicio 4 del nivel 1: el cliente se olvidó de mencionar un detalle y resulta que los valores en el diccionario original pueden estar duplicados o más, por lo cual las claves intercambiadas pueden tener duplicados. En este caso, en lugar de mostrar un mensaje de advertencia, los valores del diccionario resultante deben almacenarse como una lista. Ten en cuenta que, si el valor es único, no debe ser una lista.

```
reverse_dictionary_plus({'x': 'apple', 'y': 'banana', 'z': 'banana'})
```

```
{'apple': 'x', 'banana': ['y', 'z']}
```

```
def reverse_dictionary_plus(diccionario):
    # Creamos un diccionario vacío para almacenar el resultado
    reversed_dict = {}

    # Recorremos el diccionario original
    for key, value in diccionario.items():
        if value in reversed_dict:
            # Si ya existe el valor, agregamos la clave a una lista
            if isinstance(reversed_dict[value], list):
                reversed_dict[value].append(key)
            else:
                reversed_dict[value] = [reversed_dict[value]] + [key]
        else:
            # Si no existe, simplemente lo añadimos
            reversed_dict[value] = key

    return reversed_dict
```

✓ 0.0s

```
reverse_dictionary_plus({'x': 'apple', 'y': 'banana', 'z': 'banana'})
```

✓ 0.0s

```
{'apple': 'x', 'banana': ['y', 'z']}
```


Explicación de la función: La función `reverse_dictionary_plus` recibe un diccionario y devuelve otro donde los valores pasan a ser claves y las claves pasan a ser valores, pero manejando los duplicados. Primero, creamos un diccionario vacío llamado `reversed_dict` para ir guardando el resultado invertido. Después, recorremos cada par clave: valor del diccionario original con `items()`, que nos da acceso a ambos elementos en cada vuelta del bucle `for`. Si el valor ya existe como clave en `reversed_dict`, comprobamos si su valor es una lista con `isinstance()`. Si está, añadimos la nueva clave con el método `append()` para mantener la lista. Si no está duplicado (un valor único), lo convertimos en lista y le agregamos la nueva clave.

- 2. Conversión de tipos de datos:** El cliente recibe una lista de datos y necesita generar dos listas: una con todos los elementos que se pudieron convertir en números flotantes, y otra con los elementos que no se pudieron convertir. Ejemplo de la lista que recibe el cliente: `['1.3', 'one', '1e10', 'seven', '3-1/2', ('2',1,1.4,'not-a-number'), [1,2,'3','3.4']]`

```
conversion(['1.3', 'one', '1e10', 'seven', '3-1/2', ('2',1,1.4,'not-a-number'), [1,2,'3','3.4']])
([1.3, 10000000000.0, 2.0, 1.0, 1.4, 1.0, 2.0, 3.0, 3.4],
 ['one', 'seven', '3-1/2', 'not-a-number'])
```

```
def conversor_flotantes(lista):
    # Creamos las listas vacías para almacenar los datos:
    numeros, caracteres = [], []

    # Recorremos cada dato en la lista:
    for dato in lista:
        # Si el dato es una lista o tupla:
        if type(dato) == list or type(dato) == tuple:
            # Procesa elementos de lista/tupla
            for valor in dato:
                try:
                    numeros.append(float(valor)) # Convertimos a float y lo añadimos a la lista numeros.
                except (ValueError, TypeError):
                    caracteres.append(valor) # Si no es posible convertirlo, añadimos a la lista caracteres.
            else:
                # Mismos condicionales, pero para procesar los datos simples.
                try:
                    numeros.append(float(dato))
                except (ValueError, TypeError):
                    caracteres.append(dato)

    # Devolvemos las dos listas:
    return numeros, caracteres

1] ✓ 0.0s

lista = ['1.3', 'one', '1e10', 'seven', '3-1/2', ('2',1,1.4,'not-a-number'), [1,2,'3','3.4']]
flotantes, no_flotantes = conversor_flotantes(lista)
print('Números flotantes:', flotantes)
print('Caracteres que no se han podido convertir:', no_flotantes)

2] ✓ 0.0s

Números flotantes: [1.3, 10000000000.0, 2.0, 1.0, 1.4, 1.0, 2.0, 3.0, 3.4]
Caracteres que no se han podido convertir: ['one', 'seven', '3-1/2', 'not-a-number']
```

Explicación de la función: la función `conversor_flotantes` recibe una lista mixta que puede contener números, strings, listas anidadas y tuplas, y separa todos los elementos en dos categorías: valores convertibles a números flotantes y valores no convertibles. Primero, creamos dos listas vacías: `numeros` para guardar los valores que sí podemos convertir a float, y `caracteres` para los que no. Luego, verifica si cada elemento es una lista o tupla para procesarlos internamente, o si es un dato simple para procesarlo directamente. Para cada valor individual, utiliza un bloque `try-except` que intenta convertirlo a `float()` y lo almacena en la lista `numeros`. Si no es posible convertirlo lo guarda en la lista `caracteres`. Finalmente retorna dos listas separadas: una con todos los valores numéricos convertidos a float y otra con todos los elementos que no pudieron ser convertidos.

NIVEL 3

- 1. Contador y ordenador de palabras de un texto:** Al cliente le gustó el contador de palabras, pero ahora quiere leer archivos TXT y calcular la frecuencia de cada palabra, ordenadas dentro de las entradas habituales del diccionario según la letra con la que empiezan. Es decir, las claves deben ir de la A a la Z y, dentro de cada letra, también deben estar ordenadas alfabéticamente. Por ejemplo, para el archivo `"tu_me_quieres_blanca.txt"` la salida esperada sería:

```
{ 'a': { 'a': 3,
  'agua': 1,
  'al': 2,
  'alba': 4,
  'alcobas': 1,
  'alimenta': 1,
  'alma': 1,
  'amarga': 1,
  'azucena': 1},
  'b': { 'baco': 1,
  'banquete': 1,
  'bebe': 1,
  'blanca': 3,
  'boca': 1,
  'bosques': 1,
  'buen': 1},
  'c': { 'cabañas': 1,
  'carnes': 2,
  'casta': 3,
  'cerrada': 1,
  'con': 4,
  'conservas': 1,
  'copas': 1,
  'corola': 1,
  'corriste': 1,
  ...
  'tornadas': 1,
  'tú': 8},
  'u': { 'un': 1, 'una': 1},
  'v': { 'vestido': 1, 'vete': 1, 'vive': 1},
  'y': { 'y': 5}}
```

```
def contar_palabras_en_orden(camino_archivo):
    # Aquí vamos a guardar cuántas veces aparece cada palabra, organizada por su letra inicial:
    conteo = {}

    # Abrimos el archivo, lo leemos y convertimos en minúsculas:
    with open(camino_archivo, encoding='utf-8') as archivo:
        texto = archivo.read().lower()

    # Dividimos el texto en una lista de palabras:
    palabras = texto.split()

    # Bucle para recoger el texto y lo limpiamos con strip. Si está vacío lo ignoramos (not):
    for palabra in palabras:
        palabra = palabra.strip('!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~')
        if not palabra:
            continue

        # Guardamos la primera letra de la palabra para usarla como clave principal en el diccionario:
        primera_letra = palabra[0]

        # Organizamos por letras y contamos:
        if primera_letra not in conteo:
            conteo[primera_letra] = {}
        if palabra in conteo[primera_letra]:
            conteo[primera_letra][palabra] += 1
        else:
            conteo[primera_letra][palabra] = 1

    # Ordenamos alfabéticamente dentro de cada grupo:
    # Usase el dict(), porque la función sorted() siempre devuelve una lista, no un diccionario.
    # Pero nosotros queremos mantener la estructura de diccionario para poder acceder a los datos usando claves.
    for n in conteo:
        conteo[n] = dict(sorted(conteo[n].items())) # toma el resultado y lo convierte de vuelta en un diccionario.

    # Ordenamos las letras:
    return dict(sorted(conteo.items())) # Aquí hace lo mismo pero con el diccionario principal.
```

✓ 0.0s

```
camino_archivo = 'tu_me_quieres_blanca.txt'
contar_palabras_en_orden(camino_archivo)
```

Salida:

```

... {'a': {'a': 3,
        'agua': 1,
        'al': 2,
        'alba': 4,
        'alcobas': 1,
        'alimenta': 1,
        'alma': 1,
        'amarga': 1,
        'azucena': 1},
    'b': {'baco': 1,
        'banquete': 1,
        'bebe': 1,
        'blanca': 3,
        'boca': 1,
        'bosques': 1,
        'buen': 1},
    'c': {'cabañas': 1,
        'carnes': 2,
        'casta': 3,
        'cerrada': 1,
        'con': 4,
        'conservas': 1,
        'copas': 1,
        'corola': 1,
        'corriste': 1},
    ...
    'tú': 8},
    'u': {'un': 1, 'una': 1},
    'v': {'vestido': 1, 'vete': 1, 'vive': 1},
    'y': {'y': 5},
    'i': {'ime': 1}}

```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Explicación de la función: Dentro de la función, creamos un diccionario vacío llamado conteo, donde iremos guardando las palabras contadas. Luego, damos el comando de abrir el archivo de texto, leemos todo su contenido y lo pasamos a minúsculas. Separamos el texto en palabras con `split()`, que crea una lista con cada palabra del texto. Recorremos una por una todas esas palabras dentro de un bucle `for`. Para cada palabra, quitamos los signos de puntuación del principio y del final usando `strip('!\"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~')`. Si después de limpiarla está vacía (por ejemplo, era solo un signo), la ignoramos y pasamos a la siguiente. Guardamos la primera letra de la palabra para usarla como clave principal en nuestro diccionario. Si esa letra no tiene grupo aún en el diccionario conteo, creamos uno nuevo para ella. Si la palabra ya estaba contada antes, le sumamos 1 al contador. Si no, la añadimos con el valor 1. Después de contar todas las palabras, ordenamos alfabéticamente las palabras dentro de cada grupo de letras. También ordenamos las letras del alfabeto para que salgan en orden alfabético. Fuera de la función, definimos el nombre del archivo que queremos analizar. Llamamos a la función con ese nombre de archivo para ejecutar todo el proceso.

Para cada palabra limpia, la función extrae la primera letra y la usa como clave en el diccionario principal. Si es la primera vez que ve una palabra que comienza con esta letra, crea un nuevo subdiccionario para esa letra. Luego, o bien añade la palabra al conteo (si es nueva) o incrementa el conteo existente en 1 (si la palabra se vio antes). Después de procesar todas las palabras, la función

realiza dos operaciones de ordenamiento: primero, ordena alfabéticamente todas las palabras dentro de cada grupo de letras, luego ordena los grupos de letras mismos alfabéticamente.